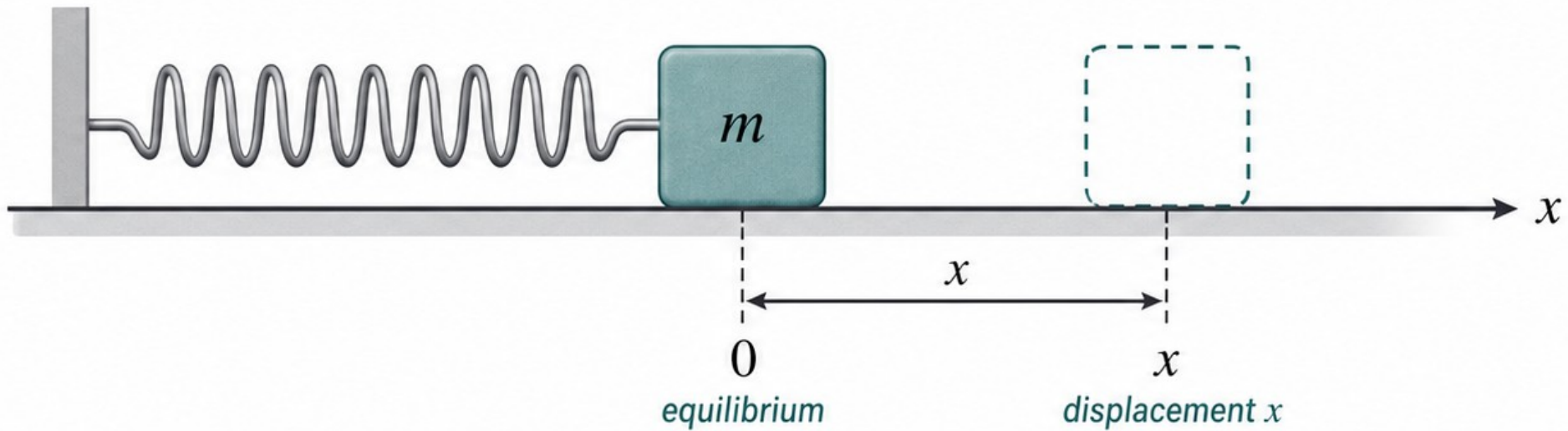


Reliable computational workflow through physics.

Week 2 | Functions, vectorisation, debugging, and testing.

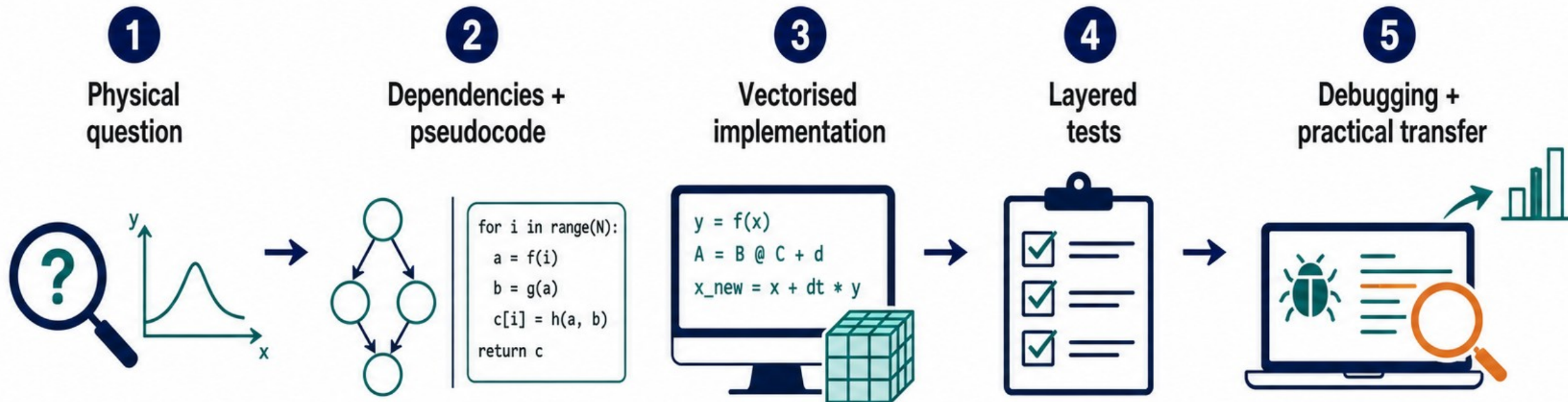


Can readable code, independent tests, and physical evidence make a calculation trustworthy?

decompose | predict | implement | test | interpret

Today's arc: decompose → predict → implement → test.

A compact physics model becomes a defensible computational result.



Preserve units



align arrays



classify defects



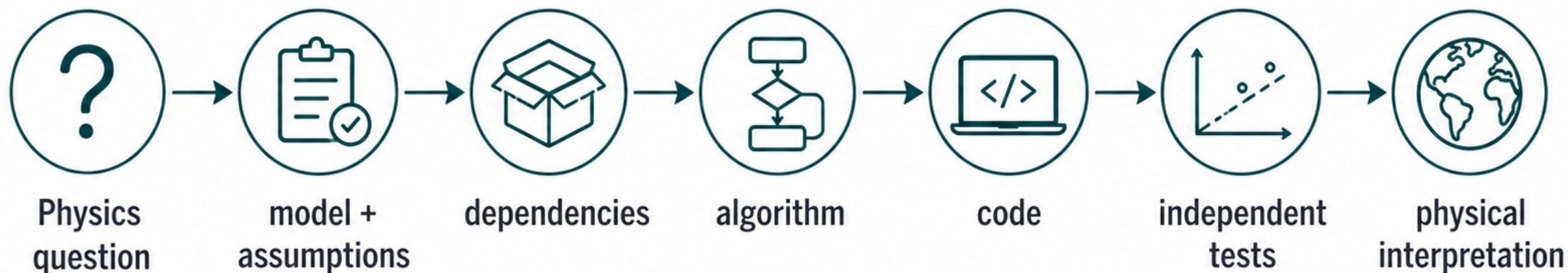
design tests



explain physical evidence

A trustworthy program is an argument.

Running code is only one claim in a longer chain.



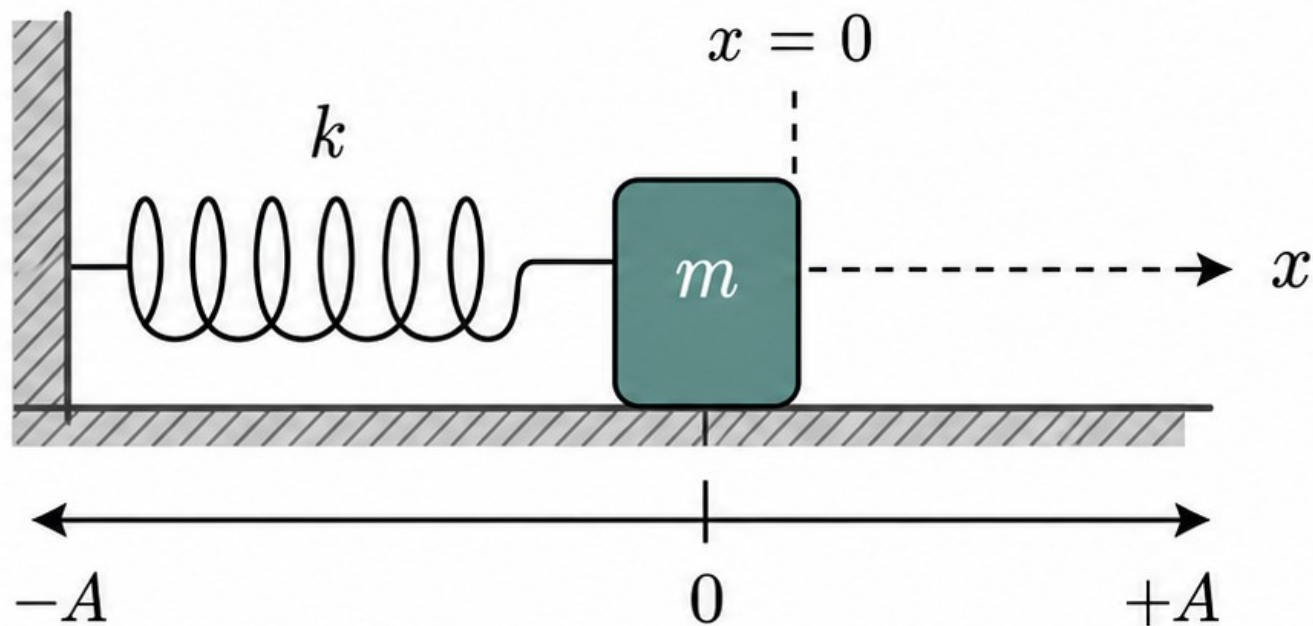
Each arrow is a decision that can be inspected.



A plausible graph is not yet evidence.

Worked model: an ideal mass–spring oscillator.

Known physics gives us strong reference behaviour.



$$m \frac{d^2 x}{dt^2} = -kx$$

$$x(t) = A \cos(\omega t + \phi)$$

$$v(t) = -A\omega \sin(\omega t + \phi)$$

$$\omega = \sqrt{\frac{k}{m}}$$

$$T = \frac{2\pi}{\omega}$$

$$m = 0.50 \text{ kg}$$

$$k = 8.0 \text{ N/m}$$

$$A = 0.12 \text{ m}$$

$$\phi = 0$$

Assumptions: linear spring, point mass, no damping, horizontal motion, equilibrium at $x = 0$.

Map symbols to names, meanings, and units.

Readable names keep dimensions visible while the model becomes code.

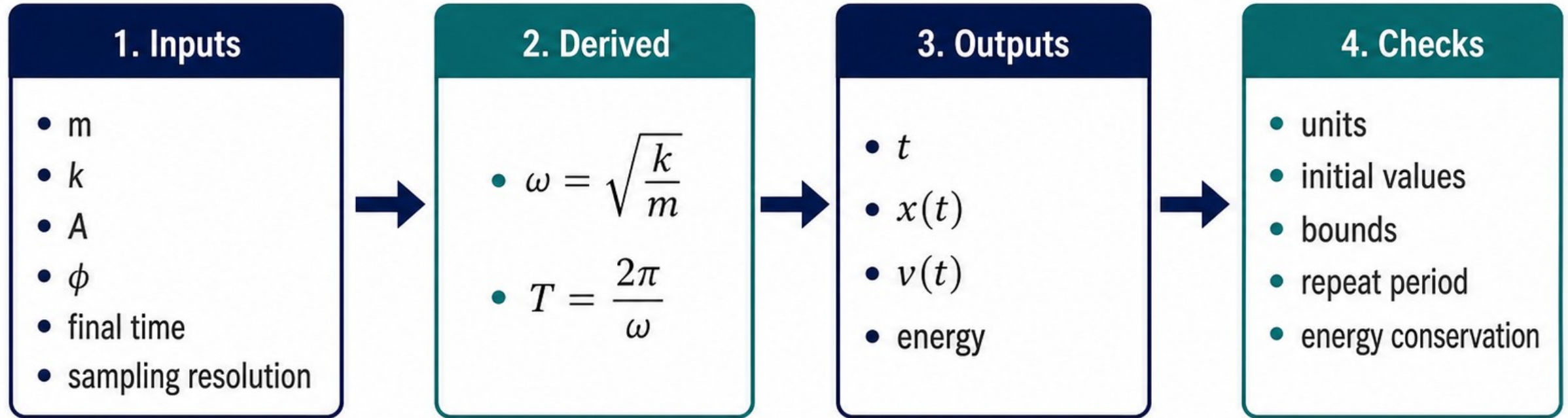
SYMBOL		READABLE NAME	UNITS
	m	mass_kg	kg
	k	springConstant_N_per_m	N/m
	A	amplitude_m	m
	ω	omega_rad_per_s	rad/s
	t	t_s	s
	x, v, E	x_m, v_m_per_s, totalEnergy_J	

 $\frac{k}{m}$ units s^{-2}

 $\sqrt{\frac{k}{m}}$ units s^{-1}

Decompose before writing MATLAB.

A dependency map prevents important decisions from disappearing into syntax.



Which quantities depend on which earlier decisions?

Pseudocode is executable reasoning.

It records the algorithm before MATLAB or AI chooses the syntax.

- 1 Define physical inputs with units
- 2 Compute ω and T
- 3 Create one aligned time array
- 4 Evaluate x , v , K , U , and E at every time
- 5 Apply independent checks
- 6 Interpret the motion and state limitations



Pseudocode should preserve dependencies and decisions — not imitate MATLAB punctuation.

Vectorisation evaluates one model across an array.

The mathematics stays visible when array operations match the formula.

```
omega_rad_per_s = sqrt(springConstant_N_per_m/mass_kg);  
t_s = linspace(0,4*period_s,1001);  
x_m = amplitude_m.*cos(omega_rad_per_s.*t_s + phase_rad);  
v_m_per_s = -amplitude_m.*omega_rad_per_s.*sin(...);  
kineticEnergy_J = 0.5.*mass_kg.*v_m_per_s.^2;  
potentialEnergy_J = 0.5.*springConstant_N_per_m.*x_m.^2;  
totalEnergy_J = kineticEnergy_J + potentialEnergy_J;
```

$$\mathbf{t} = \begin{bmatrix} t_1 & t_2 & t_3 & \cdots & t_n \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & x_3 & \cdots & x_n \end{bmatrix}$$

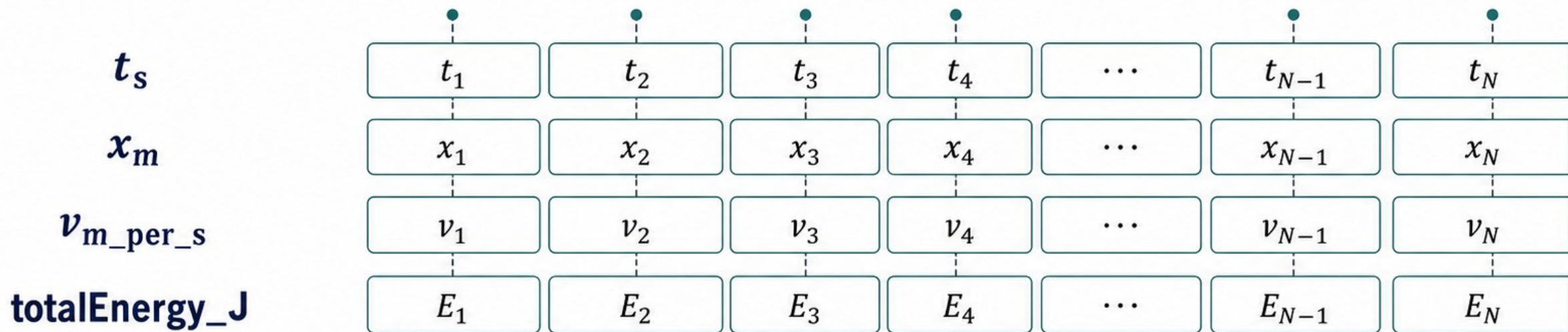
$$\mathbf{v} = \begin{bmatrix} v_1 & v_2 & v_3 & \cdots & v_n \end{bmatrix}$$

$$\mathbf{E} = \begin{bmatrix} E_1 & E_2 & E_3 & \cdots & E_n \end{bmatrix}$$

Element-wise operators preserve **one result per stored time.**

Shape contracts make array alignment testable.

One time value must correspond to one position, velocity, and energy value.



$$\text{size}(t_s) = \text{size}(x_m) = \text{size}(v_{m_per_s}) = \text{size}(\text{totalEnergy_J})$$

RISKS



1. row versus column mismatch



2. unintended implicit expansion



3. wrong colon order

CHECKS



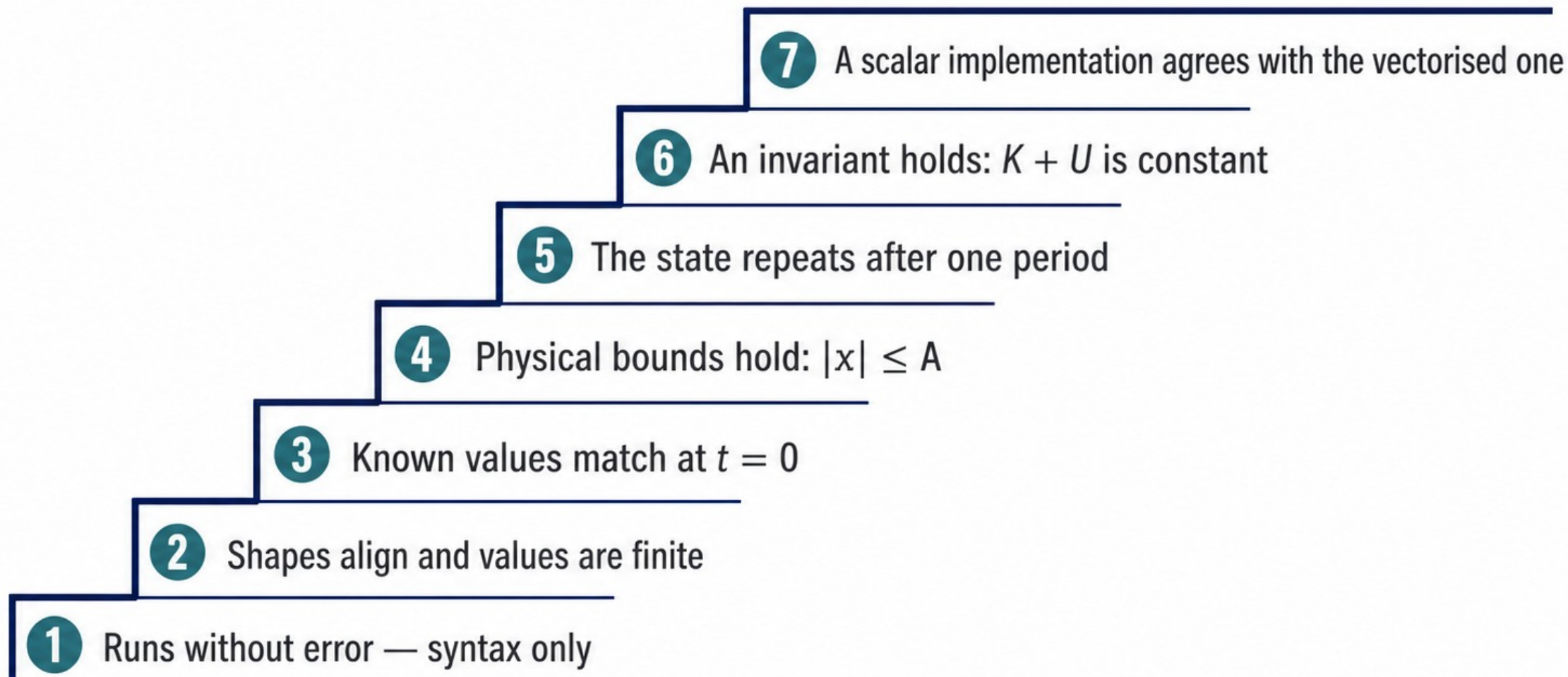
1. shape + finiteness



2. scalar loop agrees with vectorised result

Testing should climb from execution to physics.

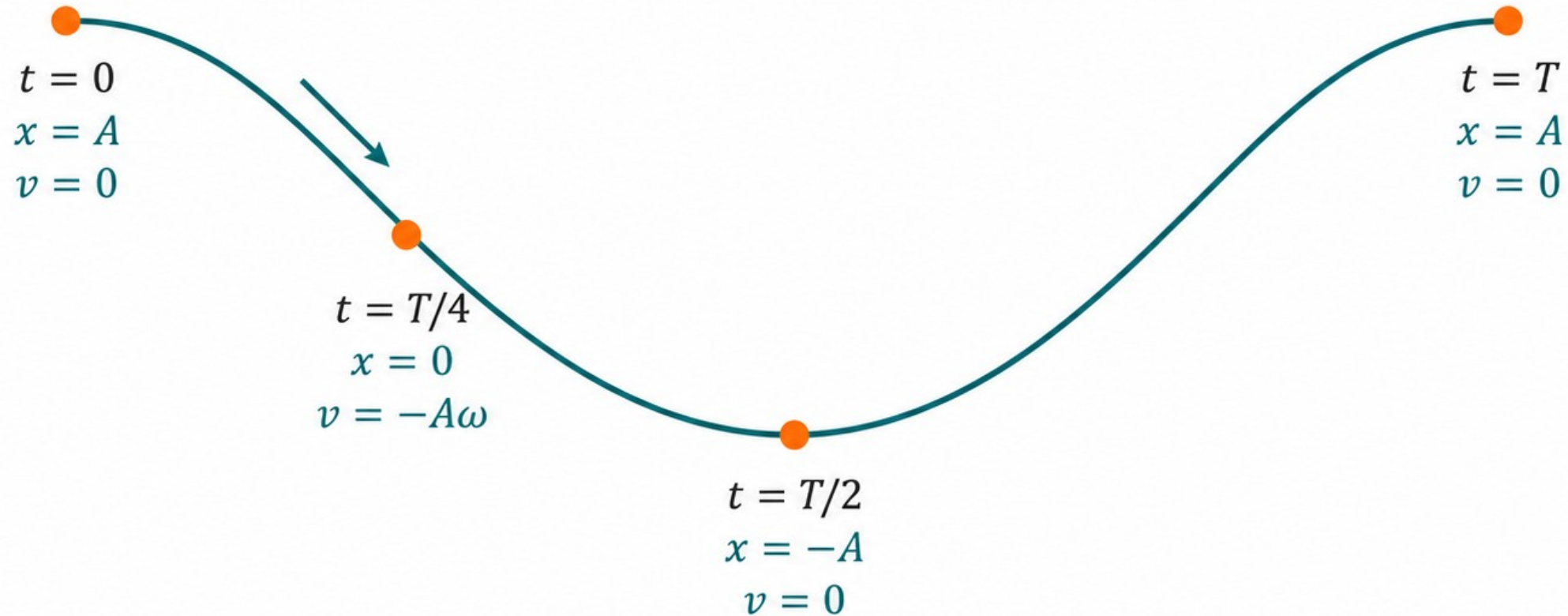
Different layers expose different classes of defect.



No single layer proves the whole model.

Known states give cheap, powerful checks.

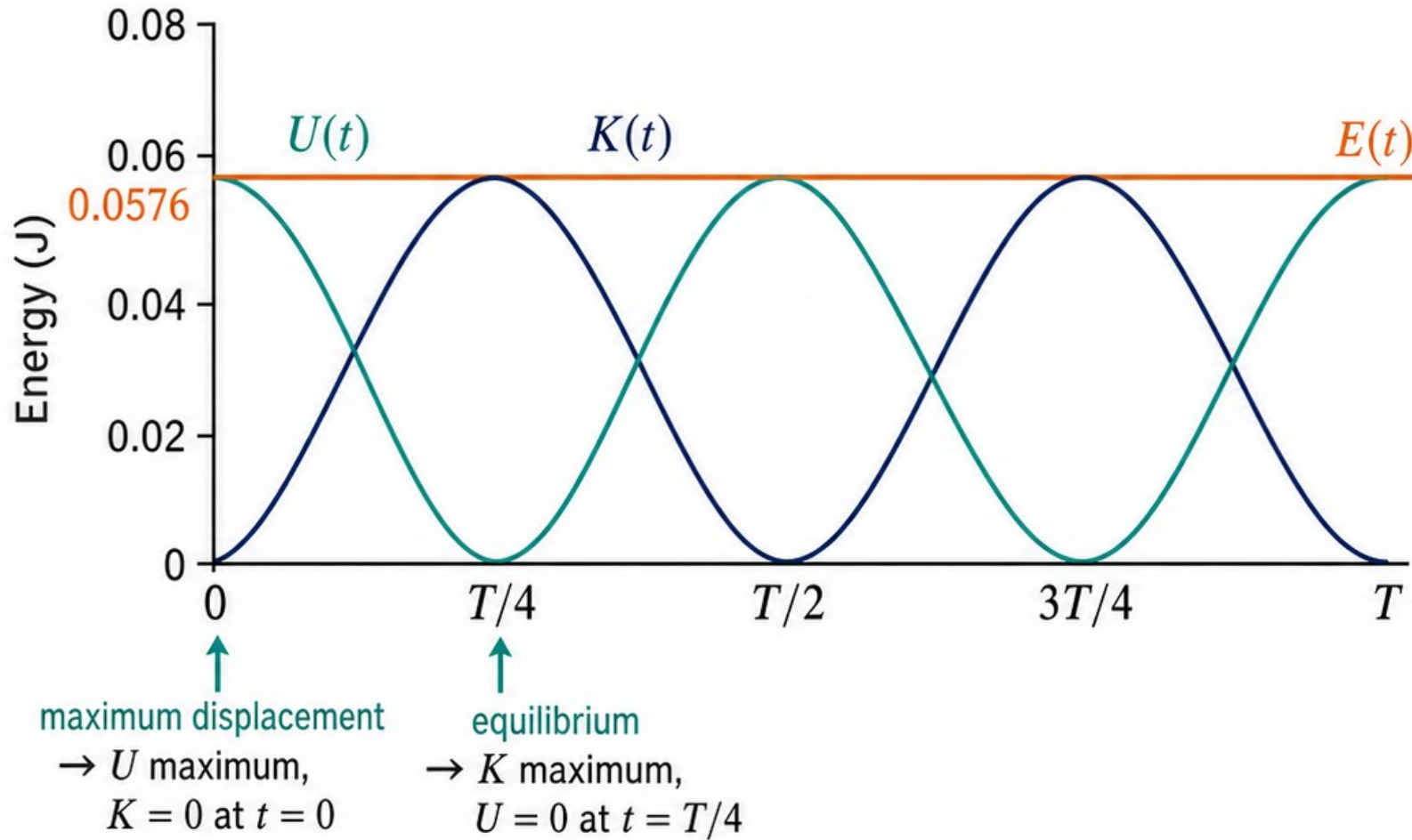
Evaluate behaviour where the physics already supplies an answer.



- Use tolerances for floating-point comparisons.
- A useful failure points toward a specific cause.

Energy conservation checks the whole trajectory.

An ideal undamped oscillator exchanges kinetic and potential energy.



$$K(t) = \frac{1}{2}mv^2$$

$$U(t) = \frac{1}{2}kx^2$$

$$E(t) = K + U = \frac{1}{2}kA^2 = 0.0576 \text{ J}$$

➡ An invariant is strong evidence — but it still has a blind spot.

One passing test is evidence — not proof.

A wrong velocity sign still preserves total energy because velocity is squared.

CORRECT

$$v(t) = -A\omega \sin(\omega t + \phi)$$



at $t = T/4$ and $\phi = 0$, $v = -A\omega < 0$



Energy test: PASS

— both velocities produce the same v^2 .

DEFECT (WRONG SIGN)

$$v_{\text{wrong}}(t) = +A\omega \sin(\omega t + \phi)$$



at $t = T/4$ and $\phi = 0$, $v_{\text{wrong}} = +A\omega > 0$



Quarter-period sign test: FAIL

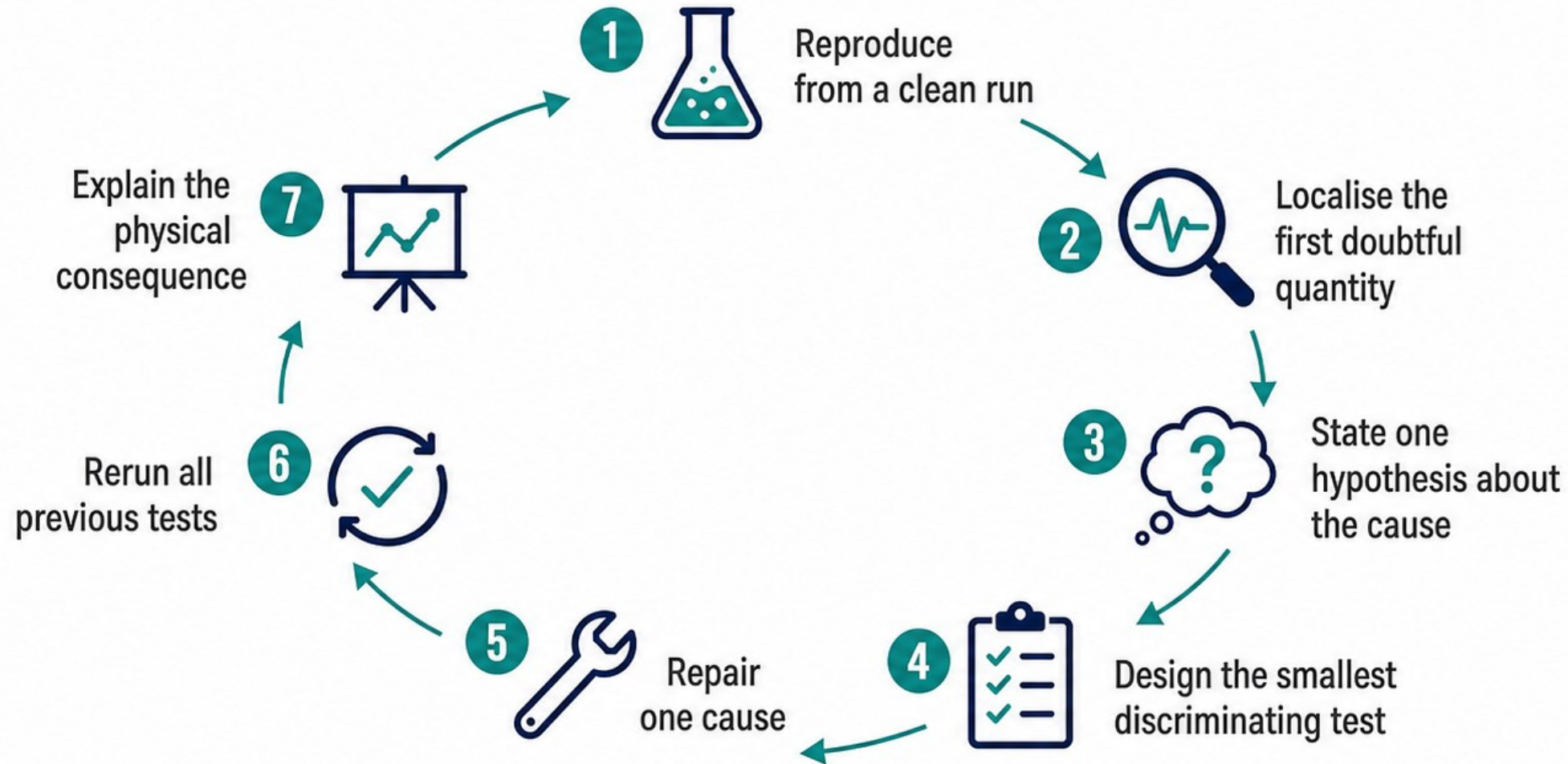
— the defect predicts the wrong direction.



Design tests that fail independently and expose different assumptions.

Debugging is another controlled experiment.

Change one suspected cause and preserve the evidence.



➡ Avoid random edits: every change should answer a hypothesis.

Diagnose plausible code by defect category.

Different defects require different evidence.

CATEGORY	CANDIDATE	FIRST TEST
Formula	<code>omega = sqrt(k*m)</code>	check units and period scale.
Logical	<code>0:tMax:dt</code>	inspect endpoints and number of values.
Array	<code>x^2</code>	inspect shapes; vectors require <code>.^2</code> .
Communication	<code>plot(v,x)</code>	compare axes with the claimed graph.
Numerical	exact equality	repeat with a justified tolerance.



Readable code exposes the scientific structure.

Compact code is not automatically clearer, faster, or more trustworthy.



READABLE WORKFLOW:



named inputs with units



derived quantities computed once



aligned arrays



checks beside the claims they support



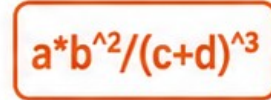
interpretation in physical language



OPAQUE WORKFLOW:



magic numbers



compressed expressions



unlabeled plots



hidden Workspace state

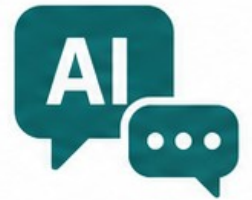
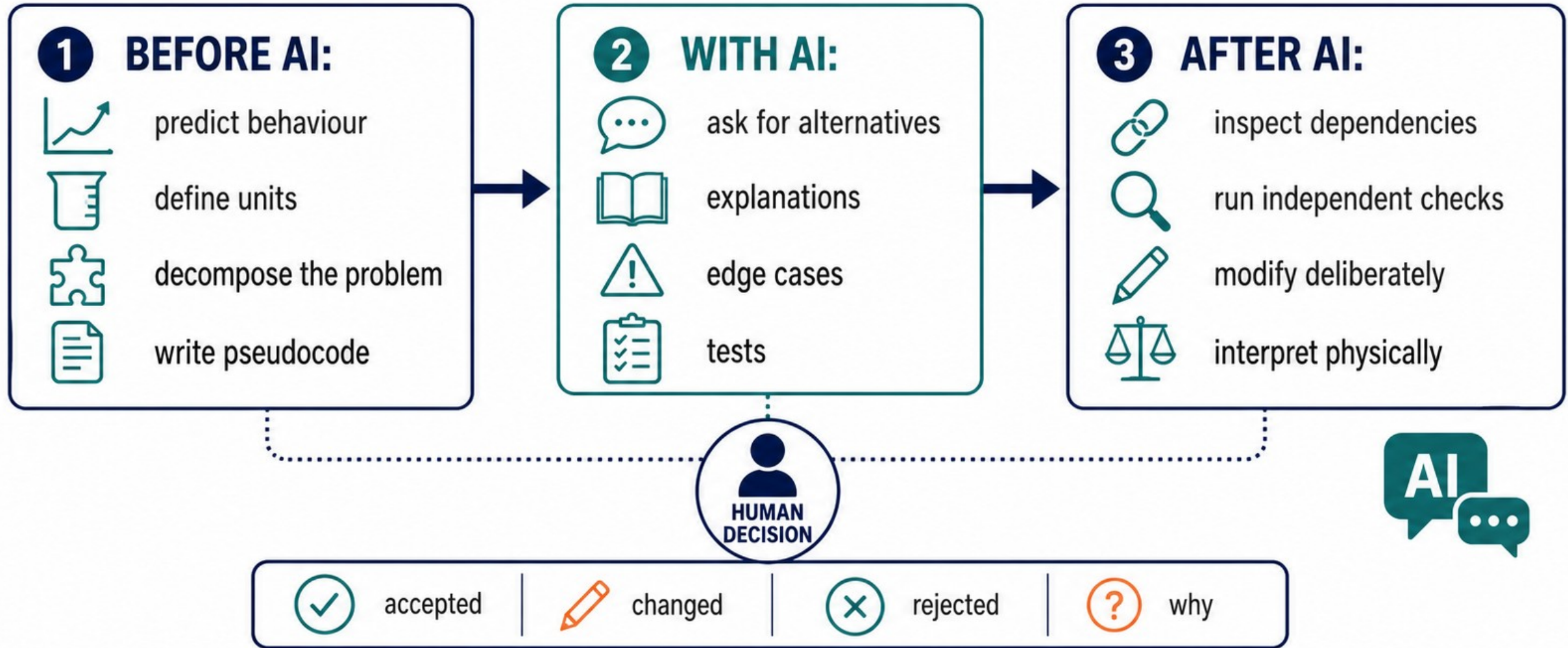


Optimise for **auditability** before cleverness.



Use AI as a critic and pair programmer — not an authority.

The student remains responsible for the model, decisions, and validation evidence.

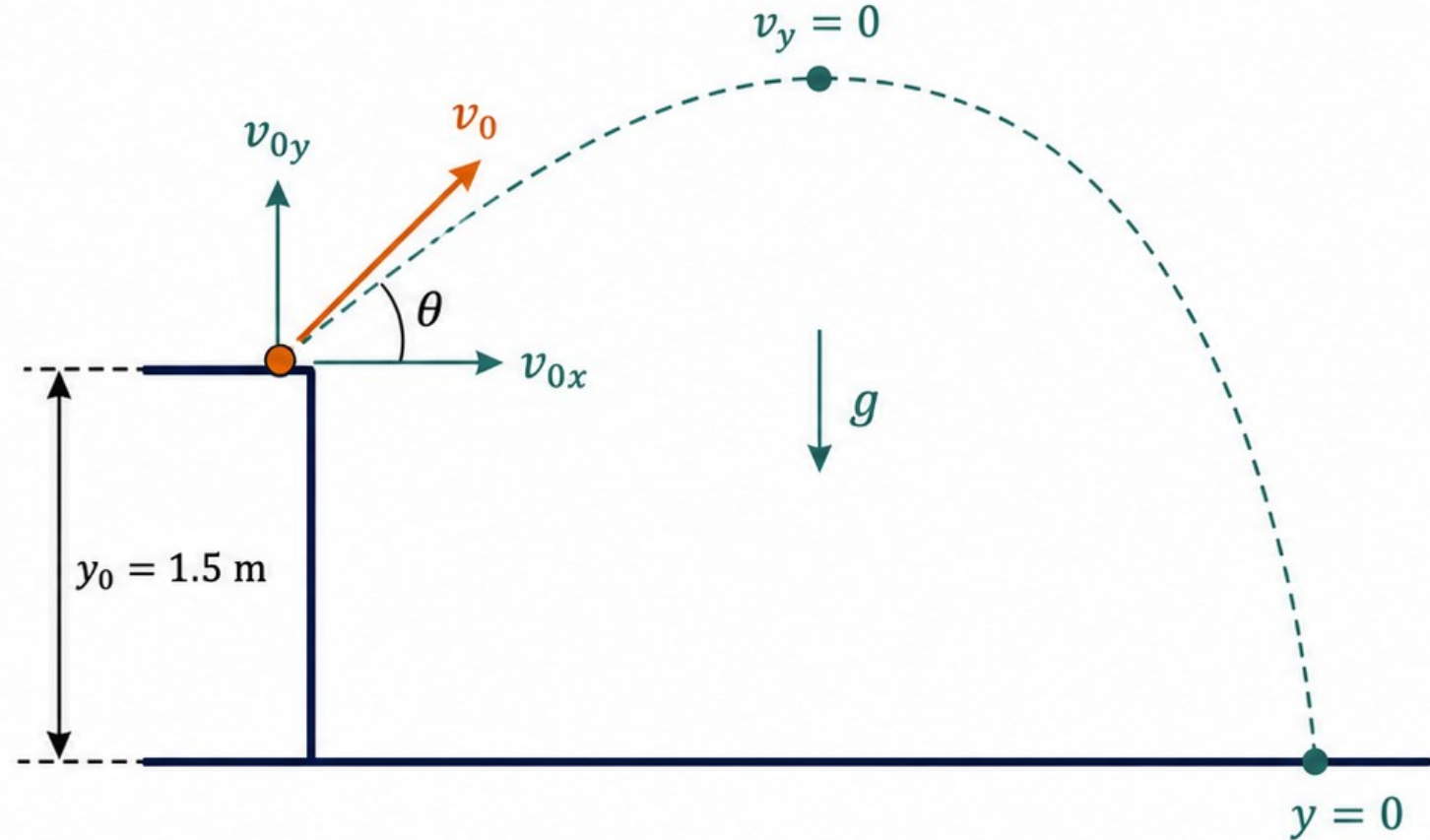


A confident explanation from AI is not validation evidence.

Practical bridge: debug and validate a projectile model.

Start individually; then make every group claim answerable with evidence.

- 1 Complete the Week 02 individual check in the single course Google Form.
- 2 Predict the trajectory and write pseudocode before using AI.
- 3 Build aligned $x(t)$ and $y(t)$ arrays.
- 4 Diagnose a plausible defective candidate.
- 5 Validate initial state, horizontal velocity, peak, range, and ground condition.
- 6 Declare AI use and the decisions your group retained.



Which oscillator checks have a projectile analogue?